

A No-Code Neural Network Approach to Telecommunications Customer Support Intent Classification for Early Churn-Risk Triage

Carl Angelo Patano
College of Computer Studies
Jose Rizal University
Mandaluyong City, Philippines
carlangelo.patano@my.jru.edu
ITC-C508

Abstract—This paper reports a controlled study of a no-code neural text classifier trained to sort telecommunications subscriber messages into three support intents: praise, complaint, and urgent account issues. The classifier was built on the Machine Learning for Kids platform, which learns entirely from tagged examples rather than hand-written conditional rules. Three models were trained on the same three classes and evaluated against one fixed set of twelve held-out phrases. The first run used the platform minimum of five examples per class. The second doubled the dataset to ten examples per class using ordinary domain sentences. The third added ten examples written specifically to address the failures observed in the first two runs. Accuracy rose from 63.64 percent to 72.73 percent when the dataset was doubled, and to 90.91 percent after the targeted additions. Doubling the data raised confidence on every class but corrected none of the four linguistic failures; on two of them the model became more confident while remaining wrong. Ten failure-targeted examples resolved three of those four failures. The results indicate that for small supervised text classifiers, how well a dataset represents the linguistic phenomena present in real input matters more than how large that dataset is.

Keywords—text classification, neural networks, supervised learning, intent detection, customer churn, training data quality, sarcasm, negation

I. INTRODUCTION

Telecommunications providers receive a continuous stream of written subscriber messages through chat windows, e-mail, social media, and support portals. These messages arrive unsorted. A message reporting a total service outage carries a very different operational urgency from a message praising a technician, yet both enter the same queue and wait for the same human reader.

The cost of this delay is not only operational. Complaint volume and unresolved urgent tickets are widely treated as leading indicators of subscriber attrition. A subscriber who reports a dead line and waits three days for a reply is measurably closer to leaving than one whose issue is routed immediately to a retention team. Automatic intent triage therefore has a direct commercial motivation: it converts an unsorted queue into a prioritised one, and it surfaces churn risk while intervention is still possible.

This exercise builds such a triage classifier and, more importantly, examines how it fails. Rather than training a single model and reporting its accuracy, three models were trained in sequence under controlled conditions and evaluated against an identical test set. The design isolates two separate questions that are frequently conflated: whether a small classifier improves because it has been given more data, or because it has been given data that better represents the input it will actually receive.

The paper is organised as follows. Section II describes the platform, the dataset, and the experimental protocol. Section III reports the results of all three runs. Section IV discusses the two debrief questions on rule-based versus example-based learning and on dataset quality. Section V addresses, in a dedicated treatment, where feature extraction occurs in this pipeline. Section VI states the limitations that remain, and Section VII concludes.

II. METHODOLOGY

A. Platform and Project Configuration

The classifier was built on Machine Learning for Kids [1], a browser-based environment that exposes supervised text classification without requiring code. The project was created under the name “Exercise PTP1 – Patano” with the project type set to text recognition. The platform presents three stages: a training stage where labelled examples are collected, a learning stage where a model is fitted to those examples, and an inference stage where new text is submitted for prediction.

Two platform behaviours shaped the experimental design. First, training will not begin unless every class holds at least five examples, which set the floor for the baseline run. Second, each trained model expires four hours after training, so all three runs were completed within a single session on 27 July 2026.

B. Domain and Class Definition

The chosen domain is telecommunications customer support. Three intent classes were defined:

Positive_Praise covers commendation of service, staff, coverage, or value. Negative_Complaint covers dissatisfaction with service quality, billing, or responsiveness where the subscriber is not blocked from using the service. Urgent_Support covers messages where service is unusable or account access is lost and the subscriber signals time pressure.

The boundary between the second and third class is deliberately narrow. Both are negative in sentiment and share vocabulary concerning signal, billing, and connection. What separates them is not polarity but urgency, and testing whether a small model can learn that distinction was one aim of the exercise.

C. Experimental Design

Three models were trained in sequence. Each run is a superset of the one before it, so no example was ever removed and the comparison is cumulative rather than independent.

TABLE I. TRAINING CONFIGURATION OF THE THREE RUNS

Run	Examples per class	Total	What was added
Phase A	5 / 5 / 5	15	Platform minimum; baseline
Phase B	10 / 10 / 10	30	15 ordinary domain sentences; no negation, no sarcasm
Step 5	12 / 15 / 13	40	10 failure-targeted sentences; 3 negation, 2 sarcasm, 3 time-pressure urgency, 2 praise

Class order is Positive_Praise / Negative_Complaint / Urgent_Support.

The separation between the second and third run is the central control of this study. Phase B increases dataset size while deliberately holding linguistic coverage constant: every sentence added was an ordinary, literal statement of the kind already present. Step 5 increases coverage rather than bulk, adding sentences that contain the specific constructions on which the earlier models failed. Any difference in outcome between the two can therefore be attributed to the character of the added data rather than to its quantity.

D. Test Protocol

One test set of twelve phrases was written before the first model was trained and was used unchanged across all three runs. No test phrase appears in any training set. The set is stratified into three case types. Nine phrases are clear, literal expressions of one of the three intents. Two phrases are sarcastic, opening with a positive marker while describing a service failure. One phrase is an administrative inquiry about updating an e-mail address, which belongs to none of the three classes and serves as an out-of-vocabulary probe. A twelfth phrase tests negation directly.

Because the out-of-vocabulary phrase has no correct label by construction, it is excluded from all accuracy computations and is reported separately as a confidence measurement. All accuracy figures in this paper are therefore computed over eleven scored cases.

E. Evaluation Metrics

For each run, a confusion matrix was constructed with actual classes as rows and predicted classes as columns. From it, precision, recall, and F1-score were computed per class, and the unweighted mean of each was taken as the macro average [2]. Overall accuracy is the proportion of correctly classified scored cases. The platform reports a single confidence percentage with each prediction, and these were recorded and analysed separately for correct and incorrect predictions.

III. RESULTS

Table II lists the prediction and confidence returned by each of the three models for every test phrase. Class names are abbreviated as Pos, Neg, and Urg.

TABLE II. PREDICTIONS ACROSS THE THREE RUNS

ID	Case type	Exp.	Phase A	Phase B	Step 5
T01	Clear	Pos	Pos 58	Pos 58	Pos 67
T02	Clear	Pos	Pos 83	Pos 97	Pos 97
T03	Clear	Pos	Pos 89	Pos 96	Pos 96
T04	Clear	Neg	Neg 74	Neg 75	Neg 92
T05	Negation	Neg	Pos 51	Pos 58	Neg 92
T06	Clear	Neg	Neg 36	Neg 65	Neg 69
T07	Boundary	Urg	Neg 42	Neg 63	Urg 77
T08	Clear	Urg	Urg 76	Urg 89	Urg 91
T09	Clear	Urg	Urg 71	Urg 88	Urg 88
T10	Sarcasm	Neg	Urg 47	Urg 50	Urg 50
T11	Sarcasm	Neg	Urg 51	Neg 49	Neg 72
T12	OOV	—	Urg 6	Urg 5	Urg 3

Values are predicted class and confidence in percent. Shaded rows are the four failure cases and the out-of-vocabulary probe. Bold entries mark failures resolved in Step 5.

Table III reports the aggregate metrics for each run. Accuracy improved at both steps, but the size of the two improvements differs sharply: doubling the dataset added roughly nine percentage points, while ten targeted sentences added a further eighteen.

TABLE III. AGGREGATE PERFORMANCE BY RUN

Metric	Phase A	Phase B	Step 5
Training examples	15	30	40
Correct (of 11)	7	8	10
Overall accuracy	63.64%	72.73%	90.91%
Macro precision	63.89%	72.22%	91.67%
Macro recall	68.89%	75.56%	93.33%
Macro F1-score	0.643	0.730	0.915
Mean conf., correct	69.57	77.13	84.10
Mean conf., incorrect	47.75	57.00	50.00
Out-of-vocab. conf.	6	5	3

Table IV gives the confusion matrix of the final model. Two of the three classes are predicted with perfect precision, and the single remaining error is a negative complaint misrouted to urgent support.

TABLE IV. CONFUSION MATRIX, STEP 5 MODEL

Actual \ Predicted	Pos	Neg	Urg
Positive_Praise	3	0	0
Negative_Complaint	0	4	1
Urgent_Support	0	0	3

The out-of-vocabulary probe is excluded.

IV. DISCUSSION

A. Explicit Rules Versus Learning From Tagged Examples

The first debrief question asks what would go wrong if this classifier were written as a chain of explicit conditional statements instead of being trained. The results of this exercise make several of those problems concrete.

The most immediate problem is that the rule set never finishes. Every keyword a developer thinks of covers only the phrasings they happened to imagine. The three classes here share almost all of their vocabulary: signal, billing, connection, and account appear across all three. A rule keyed on any of these words is useless, and a rule keyed on a distinctive word such as outage would have produced exactly the mistake the trained model made on T10.

The second problem is that rules interact badly as they accumulate. A rule mapping the word urgent to the urgent class and a rule mapping refund to the complaint class conflict the moment a subscriber writes that they urgently need a refund. Resolving such conflicts requires priority ordering, and the ordering must itself be maintained as new rules arrive. Maintenance cost grows faster than coverage.

The third problem is that rules cannot weigh evidence. A trained classifier returns a distribution and therefore signals its own uncertainty; the six percent returned for the administrative inquiry in Phase A is a usable warning that the input falls outside the trained space. A conditional statement either fires or does not. It cannot report that it is unsure, so a downstream system has no basis for deciding when to escalate to a human.

A learned model instead maps statistical associations between token patterns and labels [3]. It required no rule stating that the word refund implies a complaint; it inferred a weighting from the examples supplied. The cost of that convenience is that the model can only infer what the examples contain, which is the subject of the next section.

B. Dataset Quality, Volume, and Representative Diversity

The second debrief question asks what changed between a minimal dataset and a larger one, and why deep learning accuracy depends so heavily on data. The three-run design answers this more precisely than a two-run comparison could.

Doubling the dataset did help. Accuracy rose by just over nine percentage points, macro F1 rose from 0.643 to 0.730, and mean confidence on correct predictions rose from 69.57 to 77.13. Some of the gains were large: T06 rose from 36 to 65 percent on the same correct label. With only five examples per class, the decision boundaries were evidently drawn on very little evidence, and additional ordinary examples firmed them up.

What doubling the dataset did not do is correct a single one of the four linguistic failures. T05, T07, T10, and T11 were all wrong in Phase A, and after fifteen additional examples all four were still wrong. On two of them the situation arguably worsened. T05 moved from 51 percent wrong to 58 percent wrong, and T07 from 42 to 63. The model did not become better calibrated on these inputs; it became more confident in an incorrect answer. This is the clearest single result in the study, because it contradicts the intuition that more data reliably produces a better model.

The reason is that the fifteen added sentences contained no instance of the constructions that were causing the errors. Not one of them used a negation. Not one of them was sarcastic. The dataset grew in size while its linguistic coverage stayed exactly where it was, so the phenomena the model could not represent remained unrepresented.

Step 5 tested the alternative. Ten sentences were added, chosen not for volume but for the specific constructions observed to fail. Three of the four failures were resolved. T05 moved from 58 percent wrong to 92 percent correct, the single largest change recorded anywhere in the study, and it was produced by three sentences, as Fig. 1 shows. T07 moved from 63 percent wrong to 77 percent correct after three sentences carrying explicit time pressure (Fig. 2). T11 moved from a marginal 49 percent to a firm 72 percent.

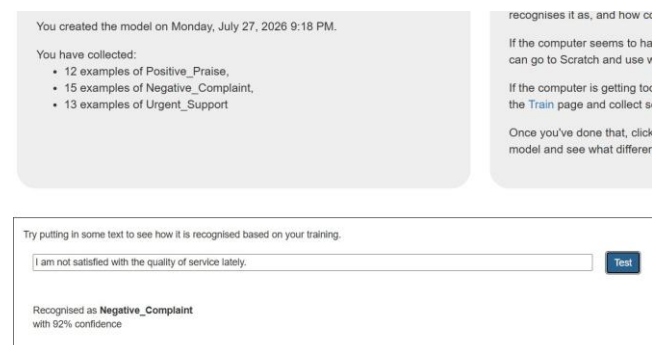


Fig. 1. Negation resolved. Test phrase T05, misclassified as praise at 51 percent in Phase A and 58 percent in Phase B, is classified correctly at 92 percent after three negation examples were added.

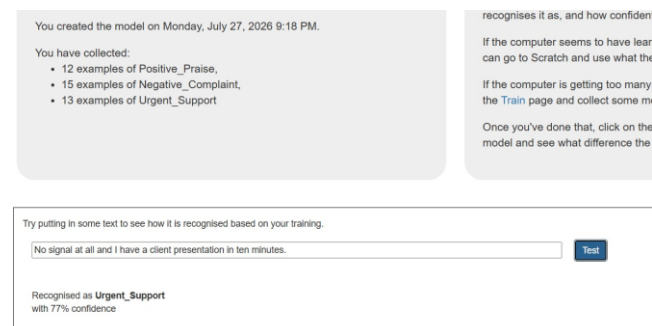


Fig. 2. Class boundary resolved. Test phrase T07, misrouted to the complaint class in both earlier runs, is classified as urgent support at 77 percent after three time-pressure examples were added.

Fifteen untargeted sentences produced a nine-point gain and fixed nothing. Ten targeted sentences produced an eighteen-point gain and fixed three of four failures. The operative variable is therefore not the quantity of training data but whether that data spans the phenomena present in real input. Volume helps only where it happens to bring diversity with it; diversity pursued directly is far more efficient.

C. Confidence as an Uncertainty Signal

In all three runs, mean confidence on incorrect predictions was lower than on correct ones. The gap was 21.82 points in Phase A, 20.13 in Phase B, and 34.10 in Step 5. The model was consistently less certain when it was wrong, which means confidence carries usable information beyond the label itself [4].

The out-of-vocabulary probe reinforces this. The administrative inquiry in T12 returned six, five, and three percent across the three runs, by a wide margin the lowest values recorded. The declining trend is itself informative: as class boundaries sharpened with better data, the model became progressively more certain that this input belonged to none of them.

For a deployed triage system this suggests a straightforward safeguard. Predictions below roughly fifty-five percent could be routed to a human agent rather than acted upon automatically. Applied to the Step 5 results, such a threshold would have caught the single remaining error, which sat at exactly fifty percent, while passing through every correct prediction except one.

V. FEATURE EXTRACTION IN THE CLASSIFICATION PIPELINE

The third debrief question asks where in the pipeline feature extraction occurs. The platform does not expose its internals, so what follows is a reasoned estimate based on the observable behaviour of the three models.

Feature extraction takes place between the training stage and the fitting of model weights, at the moment the collected text examples are handed to the training service. Raw text cannot be consumed by a neural network directly; it must first be converted into a numeric vector. The most likely sequence is tokenisation into words, normalisation of case and punctuation, construction of a vocabulary index over the training corpus, and projection of each example into a vector over that vocabulary. Each label is encoded in parallel as a target vector.

This step is invisible in the interface. The user sees a progress bar; what runs behind it is vectorisation followed by weight optimisation. Crucially, feature extraction happens only during training. At inference time the same transformation is applied to the incoming phrase using the vocabulary already fixed during training, which is precisely why out-of-vocabulary input behaves the way T12 did.

The observed failures support a bag-of-words or similar frequency-based representation [5] rather than a contextual one. The behaviour of T05 is the strongest evidence. The phrase begins with the words I am not satisfied, and the model classified it as praise at 51 percent. Under a representation that preserved word order, the negation particle would modify the sentiment word that follows it. Under a representation that discards order, the tokens not, satisfied, quality, and service are each simply present, and two of them carry strong positive weight from the training corpus. The negation is not lost as a token; it is lost as a relation [7].

The sarcasm results point the same way. In T10 the positive marker wonderful and the negative marker outage

both appear. A model reading tone would resolve the contradiction in favour of the complaint. A model summing token weights simply lets the heavier token win, and outage carried more weight toward the urgent class than wonderful carried toward praise. The prediction sat at fifty percent in every run, which is close to what one would expect when two roughly balanced signals compete [8].

This also explains why targeted examples worked. Adding negation sentences did not teach the model syntax. It gave the token not a strong association with the complaint class, so the token began to carry its own weight rather than relying on a relation the representation cannot encode. The fix operates within the limits of the feature space rather than extending it, which is why it succeeded on some cases and not on others.

Modern contextual architectures address this at the representational level by producing embeddings that vary with surrounding words [6], allowing negation and irony to be encoded structurally rather than approximated through additional examples. No-code platforms of the kind used here trade that capability for accessibility.

VI. LIMITATIONS AND FUTURE WORK

One failure was not resolved. T10 was classified as urgent support at exactly fifty percent in all three runs, entirely unmoved by the sarcastic examples added in Step 5, as Fig. 3 shows. The reason appears to be a mismatch of vocabulary rather than a failure of the strategy: the two sarcastic sentences added concerned dropped calls and data depletion, while T10 concerns an outage, a token already strongly bound to the urgent class through several training examples. The remedy would be sarcastic examples built around outage vocabulary specifically, which suggests that targeted data repair must match not only the construction but the lexical field in which it occurs.

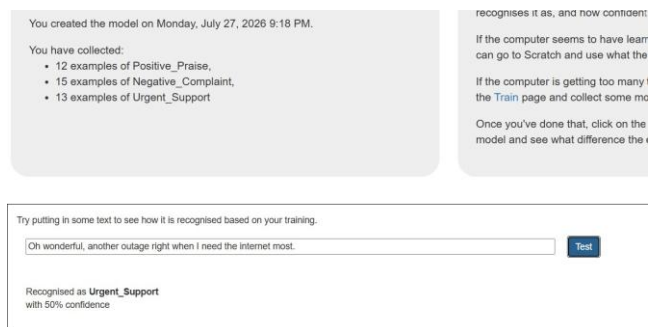


Fig. 3. Unresolved sarcasm. Test phrase T10 remains misclassified as urgent support at exactly 50 percent in the refined model, unchanged from the two earlier runs.

The study is limited in several further respects. The test set contains eleven scored cases, so a single changed prediction moves accuracy by roughly nine percentage points and no confidence interval is meaningful. All three classes are near-balanced, whereas real support queues are heavily skewed toward complaints. All text is English, while Philippine subscriber messages are frequently written in mixed Filipino and English. No administrative-inquiry class exists, so a common real-world message type has nowhere to go. The platform reports only the winning class and its confidence, so the full probability distribution across classes could not be examined.

Future work would add a fourth administrative class, expand the test set to a size that supports significance testing, introduce code-switched Filipino-English examples, and

compare the no-code result against a transformer baseline fine-tuned on the same data to quantify what the accessibility of the platform costs in accuracy.

VII. CONCLUSION

A three-class intent classifier for telecommunications support messages was built without code and evaluated across three controlled training runs against one fixed test set. Accuracy improved from 63.64 percent to 72.73 percent when the dataset was doubled, and to 90.91 percent after ten failure-targeted examples were added.

The two improvements were not equivalent. Fifteen additional ordinary sentences raised confidence throughout but corrected none of the four linguistic failures, and on two of them made the model more confident while still wrong. Ten sentences chosen to address those specific failures resolved three of the four. For small supervised text classifiers, the practical conclusion is that a dataset should be assessed by what it covers rather than by how many rows it contains, and that error analysis is a more efficient route to improvement than indiscriminate data collection.

Confidence scores were found to be systematically lower on incorrect predictions in every run, and lowest of all on input belonging to no trained class. A confidence threshold is therefore a practical safeguard for a deployed triage system, allowing uncertain messages to be routed to a human rather than misclassified silently.

REFERENCES

- [1] D. Lane, "Machine Learning for Kids," machinelearningforkids.co.uk. [Online]. Available: <https://machinelearningforkids.co.uk>
- [2] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [3] Y. Goldberg, "A primer on neural network models for natural language processing," *Journal of Artificial Intelligence Research*, vol. 57, pp. 345–420, 2016.
- [4] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [5] Z. S. Harris, "Distributional structure," *Word*, vol. 10, no. 2–3, pp. 146–162, 1954.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [7] M. Taboada, J. Brooke, M. Tofiloski, K. Voll, and M. Stede, "Lexicon-based methods for sentiment analysis," *Computational Linguistics*, vol. 37, no. 2, pp. 267–307, 2011.
- [8] A. Reyes, P. Rosso, and T. Veale, "A multidimensional approach for detecting irony in Twitter," *Language Resources and Evaluation*, vol. 47, no. 1, pp. 239–268, 2013.